Aarhus university

Computer-Science

Numerical Linear Algebra

Handin 4

Author
Søren M. Damsgaard

Student number
202309814

March 2, 2026

```python
    import numpy as np
import matplotlib.pyplot as plt

#   DISCLAIMER: i know defining V by transposing the transposed matrix is a bit cringe, but it was 'in character' for me to do it.
#---- Defining V and V^T ----
v_0 = np.vstack((1,-1,1,-1))
v_1 = np.vstack((1,1,-1,-1))
v_2 = np.vstack((0,2,2,0))
V_trans = np.vstack((v_0.T,v_1.T,v_2.T)) # 3x4 matrix
V = np.transpose(V_trans) # 4x3 matrix
print ('V^T =\n',V_trans)
print ('V =\n',V)

#---- Defining the Gram  Matrix ----
Gram = V_trans @ V # 3x3 matrix
#   From definition 8.11 we have that if the inner product between two vectors is 0 for <u_j,v_i> where j != i,,
#then the vectors are orthogonal.
#   All entries in a gram matrix is the inner product between two vectors, so if all entries except the diagonal are 0,
#then the vectors are orthogonal.
print('Gram =\n',Gram)


#---- Defining the projection matrix ----
#   Project x on the span(samlingen) of v_0, v_1, v_2 (see V_trans for vectors respectively)
x = np.vstack((4.0,3.0,2.0,1.0))

#   Knowledge from definition 8.19 and the Gram matrix.
P_x = (np.vdot(x,v_0))/Gram[0,0]*v_0 + (np.vdot(x,v_1))/Gram[1,1]*v_1 + (np.vdot(x,v_2))/Gram[2,2]*v_2
print('Projection of x on the span of v_0, v_1, v_2 =\n',P_x)
#   This represents the point in the span (subspace) of v_0, v_1 and v_2 that is closest to x

#   Distance from the projection to x
distance = np.vdot(x-P_x,x-P_x)
print('Distance from the projection to x =\n',distance)
#   This represents the smallest/closest distance from the point x to the subspace

#---- v_3 = x-P_x othogonal to v_0, v_1 and v_2 ----
#   Remember if the dot product between two vectors are 0 they are orthogonal.
v_3 = x-P_x
print('v_3 = x-P_x =\n',v_3)
```

```
print('v_0 is orthogonal to x-P_x:', np.vdot(v_0,v_3))
print('v_1 is orthogonal to x-P_x:', np.vdot(v_1,v_3))
print('v_2 is orthogonal to x-P_x:', np.vdot(v_2,v_3))

#---- orotnormal basis for the span of v_0, v_1, v_2, v_3 in R^4 ----

# New tech discovered for making matrix... no way i'm gonna change the earlier code...#Badboy
newV = np.hstack((v_0,v_1,v_2,v_3)) # 4x4 matrix
print('New V =\n',newV)
newV_trans = np.transpose(newV) # 4x4 matrix

newGram = newV_trans @ newV # 4x4 matrix
print('New Gram =\n',newGram)
# We see they are orthogonal but not ortonormal, lenght 1 and jazz, so we need to noralize them with Lemma 8.15
# Which says that if we have an orthogonal basis we can make it orthonormal by dividing each vevtor by its own lenght.
v_0_norm = v_0/np.linalg.norm(v_0)
v_1_norm = v_1/np.linalg.norm(v_1)
v_2_norm = v_2/np.linalg.norm(v_2)
v_3_norm = v_3/np.linalg.norm(v_3)
newV_norm = np.hstack((v_0_norm,v_1_norm,v_2_norm,v_3_norm)) # 4x4 matrix
print('New V normalized =\n',newV_norm)
newV_norm_trans = np.transpose(newV_norm) # 4x4 matrix
newGram_norm = newV_norm_trans @ newV_norm # 4x4 matrix
print('New Gram normalized: G = V^T V = I_4 =\n',newGram_norm)
# The new Gram matrix has become the identity matrix which means according to Lemma 9.8 it's Orthonormal.
# and since its orthonormal for R^4, and has 4 vectors, it's the basis for R^4 according to definition 9.10.
```

Appendix

Clearly a misguided appendixed left redundant after its creator realized that the code was not that long and could be easily included in the main text without a kerfuffle.